

A Deep Reinforcement Learning Approach for Composing Moving IoT Services

Azadeh Ghari Neiat, Athman Bouguettaya, *Fellow, IEEE*, Mohammed Bahutair

Abstract—We develop a novel framework for efficiently and effectively discovering crowdsourced services that *move* in close proximity to a user over a period of time. We introduce a moving crowdsourced service model which is modelled as a moving region. We propose a deep reinforcement learning-based composition approach to select and compose moving IoT services considering quality parameters. Additionally, we develop a parallel flock-based service discovery algorithm as a ground-truth to measure the accuracy of the proposed approach. The experiments on two real-world datasets verify the effectiveness and efficiency of the deep reinforcement learning-based approach.

Index Terms—IoT, mobile crowdsourcing, mobile IoT services, moving crowdsourced service, service composition, deep reinforcement learning, MapReduce, crowdsourced IoT service, spatio-temporal mapper, mobile computing.

1 INTRODUCTION

The crowdsharing economy is an emerging dynamic ecosystem where people create new on-demand services through sharing or exchanging resources to achieve mutually beneficial goals [1]. This new type of economy has the potential to be applied in a diverse range of sectors, including tourism and hospitality, labor and service platforms, mobility and logistics [2]. Two well-known examples of are Uber¹ and Airbnb². The foundations of this emerging economy are anchored in *crowdsourcing* and *crowdsharing* [3]. In crowdsourcing environments, crowds need a medium to interact and produce results. A wide variety of devices are used to facilitate certain types of crowdsourcing [4], [5]. In particular, Internet of Things (IoT) devices are usually used to allow the crowd to provide and use *services*. We define *services* as an abstraction that transforms IoT data into *actionable information* [6]. In that respect and more formally, a *service* is defined by its functional and non-functional attributes. A functional attribute is usually defined as *what* a service provides, i.e., the purpose of the service. Non-functional attributes are *qualities* attached to service provisioning, i.e., Quality of Service (QoS). For example, a functional attribute of an airline service is *reservation*. A non-functional attribute of this service is the *price* which would be attached to the reservation.

A *moving crowdsourced IoT service* is a service provided by an IoT device *moving* in *time*, *space*, or both. An exam-

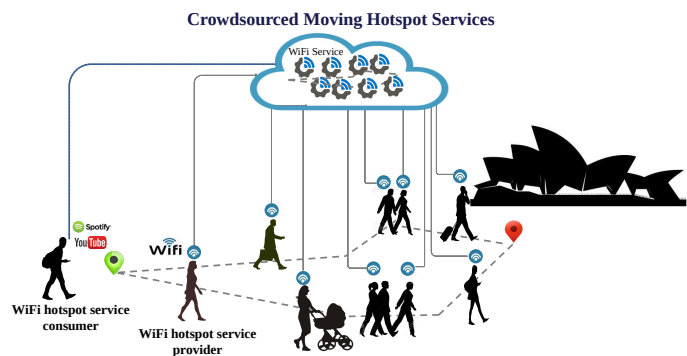


Fig. 1. WiFi Hotspot Sharing Scenario.

ple of such services is a WiFi hotspot provided by a person through their smartphone. This type of crowdsourced IoT services is characterised by their *spatio-temporal* aspects. The *spatio-temporal* properties refer to the location/space and time/period in which crowdsourced services are provisioned and consumed. We use *moving IoT service* and *mobile IoT service* interchangeably.

Mobility is an important and intrinsic part of the non-functional aspect of crowdsourced services. The mobility of IoT devices provides opportunities to dynamically extend service coverage to larger spaces and at flexible times. Mobility, however, presents key challenges in terms of qualitative factors (e.g., availability) if the aim is to provide users with the best quality of experience. In this respect, we focus on the spatio-temporal aspects as key parameters to query moving crowdsourced services.

We identify two key types of crowdsourced services with regard to spatial location: *fixed* and *moving*. A fixed crowdsourced service refers to services which are permanent in *space* during the time period of the service provisioning. For example, Diana may share her WiFi hotspot while she is sitting at a coffee shop. In contrast, moving crowdsourced service is not tied to any specific location at any point in time. For example, Diana may share her WiFi hotspot as

• A. Ghari Neiat is with the School of Information Technology, Deakin University, Geelong, VIC 3220, Australia Email: azadeh.gharineiat@deakin.edu.au.

A. Bouguettaya and M. Ba-Hutair are with the School of Computer Science, University of Sydney, Darlingtown, NSW 2008, Australia. E-mail: athman.bouguettaya@sydney.edu.au and mbah6158@uni.sydney.edu.au.

1. www.uber.com

2. www.airbnb.com

she moves from one location to another when strolling in the city. More specifically, we assume that a *fixed* hotspot service will remain available (in terms of time and location) when selected by a service consumer and throughout the provisioning of the service. However, for *moving* hotspot services, the availability and location can change during service provisioning. Additionally, crowdsourced services have another dimension, i.e., they may also be *deterministic* or *non-deterministic*. A crowdsourced IoT service is *Deterministic* if *time period* and availability at a certain location are *known in advance*. A crowdsourced IoT service is *Non-deterministic* if time period and location availability are *not known in advance*.

There has been a large body of research on service selection and composition in mobile environment [7], [8], [9], [10]. Most existing methods focus on service selection and composition based on Quality of Service (QoS) parameters [11] and energy consumption [12], [13]. They do not take into account the mobility of a user or service provider. There are only a few works that address the mobility-aware service selection problem [14], [15]. However, these works do not consider the composition of mobile IoT services while both IoT service provider and consumer are moving together.

We identify the following research challenges. The first challenge is **connectivity** which is an intrinsic part of moving crowdsourced service discovery. A *moving service should stay connected* with a service provider, i.e., be within connectivity proximity of the provider. This requires determining co-movement patterns. We propose a parallel flock-based service discovery to find co-movement services. The parallel flock-based approach is based on a spatio-temporal MapReduce to efficiently find flock patterns [16]. We first apply a *temporal map* step to prune sub-trajectories of moving services with regards to a user trajectory. We then deploy a *spatial map* step to filter candidate moving services which are located within a circular region of a user trajectory. The second challenge is **service continuity** to connect to the next moving service as an IoT service provider and a user is not necessarily sharing all their route. Therefore, *an effective composition approach is required to select an optimal set of available moving crowdsourced services which ensure the service continuity*. Most existing trajectory similarity joins approaches are time-interval based [17], [18] and their methods are not applicable to continuous temporal matching as they retrieve approximate results. Little work [19] addresses the issue of continuous nearest neighbour joins on big trajectory datasets. The key difference with our approach is the need to select continuous sub-trajectories considering a range of QoS parameters. We propose a Deep Reinforcement Learning-based composition algorithm to find and compose valid candidate moving services which overlap with the user trajectory. Third challenge is **indexing**. Existing co-movement discovery methods usually rely on *centralized index methods* like R-tree. [20] shows that the performance of the existing co-movement discovery methods like flock [16], convoy [21], swarm [22], group [23] and platoon [24]

degrade dramatically as the dataset scales up. As a result, creating and maintaining an index in parallel computing like MapReduce cannot be effective [20]. To address this we propose a pruning approach that aims at diverting the algorithm from selecting invalid services without the need to index them.

The contribution of the paper is summarized as follows:

- We propose a spatio-temporal model for *moving crowdsourced IoT services*. Our previous work in [25] and [26] proposed a selection and composition model for *fixed* crowdsourced services based on spatio-temporal features. We also proposed a temporal non-deterministic service discovery approach. In this work, we focus on *deterministic moving services*.
- We design a novel solution based on deep reinforcement learning to support effective discovery and composition without using an index.
- We devise a ground-truth, called *parallel flock-based moving crowdsourced service discovery*, using Apache Spark. This is used to measure the accuracy of the proposed discovery model in terms of service discovery in each timestep. The algorithm is based on a spatio-temporal MapReduce. After the spatio-temporal filtering is conducted, we retrieve the valid candidates and feed them to our deep reinforcement learning-based composition approach.
- We conduct a set of extensive experiments on two real datasets. The results show the efficiency and effectiveness of the approach in terms of accuracy, learning speed and scalability in comparison with the ground-truth.

The rest of the paper is organized as follows: Section 2 surveys related work. Section 3 introduces our system model and states the problem of crowdsourced moving service composition. Section 4 presents the deep reinforcement learning-based composition algorithm. Section 5 provides the ground-truth approach. Section 6 reports our experimental results. Section 7 concludes the related work and highlights future work.

Motivation Scenario

Scenario 1: WiFi tethering may be a connection option when free public WiFi is not available or effective due to low connection speed and limited capacity. In WiFi tethering, the crowd can switch on their IoT devices' WiFi hotspot and share their data balance to other devices for some rewards. As a result, WiFi hotspot can be crowdsourced. IoT devices can be anything the crowd has and is connected to the Internet. Generally, IoT devices can be divided into two main categories: fixed and mobile. Fixed IoT devices are referred to devices that typically do not move, e.g., smart fridge, smart TV, etc. Mobile devices are devices that are inherently made to be carried by people, such as smartwatches, and smartphones. Wearables are a subset of mobile IoT devices. They are special IoT devices that are meant to be worn (e.g., smart shoes and smart

shirts). It is predicted that wearables in the near future would become ubiquitous [27].

We consider WiFi hotspot sharing as one of the representatives of crowdsourced IoT services. For instance, it can be leveraged to offer a range of crowdsourced services such as WiFi-coverage travel planning for cost-effective media streaming. In this regard, crowdsourced WiFi hotspot services are provided by smartphones which are moving in space and time through a mobile application. For example, Open Garden³ created an application that lets users share and consume the Internet among each other. They use a monetary incentive, where providers set the price per MB and consumers pay according to their usage.

The trajectories of the services are deterministic. Deterministic trajectories refer to the a-priori knowledge of the trajectories. It is assumed that there is a platform that incentivizes WiFi hotspot providers to move to specified areas and share their resources [28]. Therefore, the WiFi hotspot providers are assumed to follow certain trajectories that have been assigned. In particular, we focus on the trajectories of *pedestrians*. We also assume that WiFi hotspot services will overlay digital maps. We propose to reformulate the research problem of *moving crowdsourced IoT service selection and composition as finding the optimal composition of WiFi hotspot moving services which provide the best quality of experience to fulfill users' specific requirements/expectations (e.g., watching online videos and signal strength)* (Fig. 1).

Scenario 2: Providing convenient power to IoT users is a valuable service to help them to stay connected. Wireless energy transfer technologies [29] transforms the way people charge their IoT devices and enables energy sharing between mobile IoT devices *seamlessly* from a distance. We consider *crowdsourcing energy as a service* scenario which has the potential to create a *green* environment [30]. For example, someone wearing smartshoes that have generated energy through walking can share the harvested green energy with other IoT devices wirelessly within a *range*. This can also enable users to recharge their smartphones as they move.

2 RELATED WORK

We provide an overview of the relevant research in relation to the selection and composition of crowdsourced moving services. We first present a review of *mobile crowdsourcing* frameworks. We then review previous studies on *service composition*. Finally, we survey *co-movement discovery* approaches.

2.1 Mobile Crowdsourcing

Crowdsourcing aims at *outsourcing* tasks to the *crowd* for faster and efficient execution. Crowdsourced tasks may require specific requirements compared to others. For example, computing tasks require the crowd to have computers with enough processing power. *Spatial tasks*

are tasks that require specific spatial attributes from the crowd for successful task fulfillment. For example, spatial tasks may require the crowd to be physically located at certain locations and collect data. Others may require the crowd to move between different locations to fulfill their task. Crowdsourcing spatial tasks are often referred to as *Mobile Crowdsourcing*. Mobile crowdsourcing typically involves a crowd with mobile devices (e.g., smartphones and smartwatches) to satisfy the spatial requirements by the tasks. There have been several works in the area of mobile crowdsourcing. In this section we focus on two types of mobile crowdsourcing: (1) spatial crowdsourcing [5], [31], and (2) urban crowdsourcing [32], [33].

In spatial crowdsourcing, the outsourced tasks are typically designated with specific locations. The crowd should execute the tasks at the specified locations. CrowdSensing@Place (CSP) [34] is one example of spatial crowdsourcing. CSP aims at labeling places into categories (e.g., cafe or restaurants). It leverages spatial data, user trajectories, and sampled audio clips and images to achieve this. Another framework is proposed in [35] that tries to find a suitable set of users to answer location-based queries. Location-based services (e.g., Foursquare) are used to answer queries instead of relying on spatial task assignments to users. It is worth noting that despite many works proposed on spatial crowdsourcing, there has been little attention to spatio-temporal crowdsourcing.

Urban crowdsourcing aims at detecting users' transportation modes using collected spatio-temporal and acceleration data [32]. The work in [36] and [37] proposes mobile crowdsourcing approaches that use commuters trajectories to predict real-time arrival of buses. A new platform, namely OneBusAway, is proposed in [38] to predict the real-time arrival time of buses. OneBusAway collects users' comments and feedback through Twitter, blogs and bug trackers. Another approach is proposed in [39] to accurately identify the right crowdsourced sensors to answer a particular journey planning request. In particular, an unsupervised learning approach is introduced to select and cluster the right mobile crowdsourced sensors based on common patterns in their trajectories.

We investigate in our work the concept of *mobile crowdsourced services* that combines mobile crowdsourcing and the service paradigm. We show that mobile crowdsourced services offer more efficient techniques for processing *spatio-temporal sensor data*.

2.2 Service Composition

Mobile crowdsourcing aspires to provide a platform where moving users act as service providers offering crowdsourced services for a smart city. Only a few studies have focused on crowdsourcing as a service [8], [40], [41], [42]. For example, in [8] a CrowdService framework is developed to provide crowd worker and crowd intelligence as crowd services via mobile crowdsourcing. A composition approach is developed based on Genetic Algorithm (GA) to provide near-optimal composite services. [43] introduces a

3. <https://opengarden.com>

crowdsourced service platform called sensing as a service (S^2aaS). In this platform, mobile service providers or smartphone users can request sensing services if they have fulfilled previous sensing tasks [44]. Furthermore, several crowdsourced service composition frameworks have been proposed to select and compose crowdsourced services. In [45], an agent-based crowd service composition framework is proposed in a scenario of purchasing a secondhand laptop. The framework takes into account users' constraints including the response time and the cost to select and compose crowdsourced services. A crowdsourced service framework is proposed for composing energy services [15]. A new temporal composition algorithm which is a variation of a fractional knapsack algorithm is developed to compose crowdsourced energy services to satisfy a user's energy requirement. However, the studies above assume that the crowdsourced services are fixed in space and time.

A service composition approach is proposed in [11] while considering the service's QoS and mobility. The mobility-aware QoS notion is built using the service invocations' mobility model, which describes the performance of a service. On the other hand, only one QoS criterion i.e., location-sensitive response time is taken into account to select an optimal service composition plan in [9] and [11]. A Mobile Service Sharing Community (MSSC) is proposed in [9] for moving service users and providers. Additionally, a composition approach for mobile service is introduced based on Krill-Herd algorithm for finding optimal response time. In [14], a mobility-aware service selection approach is proposed that takes into account a user's movement path. The selection algorithm is modelled based on GA and simulated annealing algorithm. The proposed approach reduces the response time of the service request by selecting appropriate edge servers as the user moves. A three-tier IoT service composition framework that takes into account spatio-temporal and energy constraints is proposed in [13]. In this framework, an IoT service composition algorithm is developed that adopts GA, Ant Colony Optimization, and Particle Swarm Optimization to find an optimal composition plan while reducing network energy consumption. In contrast to the above studies, we propose a service composition approach where both service provider and user are moving together.

There are some composition approaches that integrate service selection and compositions with Reinforcement Learning RL [46], [47], [48]. In [49], a novel multicriteria-driven reinforcement learning algorithm is proposed for dynamic Web service composition which adapts Randomized Reinforcement Learning (RRL) [50]. The proposed approach enables continuous adjustment of the service composition through learning about the quality of new services and exploring new composition plans while optimizing multiple criteria and satisfying users' constraints. There are two RL-based service composition algorithms [51], [52] which are not efficient for large scale service composition. In [53], a new model for the large-scale adaptive service composition based on Multi-Agent Reinforcement Learning (MARL) is introduced. The model utilizes the coordination

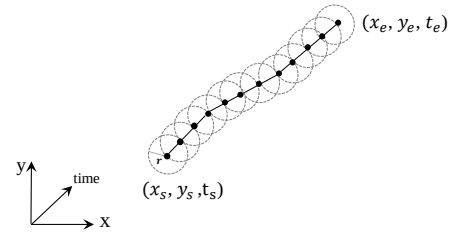


Fig. 2. Moving crowdsourced service model.

equilibrium and fictitious play process to ensure the convergence of the agent to a unique equilibrium. The previous approaches are applied in web service composition. In contrast, our approach is based on spatio-temporal aspects of moving IoT services.

2.3 Co-Movement Discovery

There exist many studies to discover similar co-movement patterns including flock [54], [16], convoy [21], swarm [22] and moving clusters [55]. A flock pattern refers to a group of at least $m > 1$ objects moving together within a user-defined disk with radius r for at least $k > 1$ consecutive timestamps. In the flock pattern discovery, moving objects are clustered based on a disk-based region. A key challenge in flock pattern is the selection of proper disk size. Larger disk size may capture wrong objects and a smaller radius may miss some objects. To overcome this challenge, a convoy pattern is introduced to cluster moving objects using a density-based clustering method like DBSCAN [56]. Instead of staying within a disk, moving objects of a convoy are connected based on the density. While flock and convoy patterns have a temporal consecutiveness constraint, swarm and moving cluster patterns relax this constraint through accepting short-term deviations. However, it is not required to have unique (i.e., same) objects throughout the timestamps in moving cluster pattern. Swarm, group [23] and platoon [24] adopt different pruning techniques relying on depth-first search which are not efficient in parallel processing [20]. Most existing methods rely on centralized indexing methods built on top of the whole dataset which may not be effective and efficient in parallel computing [20]. We focus on the parallel discovery of moving flock patterns without using an indexing method.

3 SYSTEM MODEL AND PROBLEM FORMULATION

In this section, we first formally model crowdsourced moving services by moving regions. We then formulate the problem of selecting and composing the best candidate moving services with respect to their deterministic behavior.

3.1 Problem Formulation

Definition 1: Moving Crowdsourced Service MS. A moving crowdsourced service MS is a tuple of $\langle id, F, Q \rangle$ where

- id is a unique service identifier,

- F is a function offered by MS , e.g., providing a moving WiFi hotspot. The function of the moving service represents a moving service's coverage in space and time. We define the coverage as a moving region which is described by a tuple of $\langle T_s, R_{t_i}(p_i) \rangle$ where
 - T_s is a service trajectory which is a sequence of timestamped samples $\{\langle t_i, x_i, y_i \rangle, 1 \leq i \leq k\}$, where x_i is longitude and y_i is latitude and t_i is a timestamp.
 - $R_{t_i}(p_i)$ is the specific coverage region that is offered by MS . Without loss of generality, the moving region is represented by a circular area which is centered at p_i with the radius r at t_i ,
- Q is a set of QoS attributes q_i (e.g., capacity).

Fig. 2 shows a moving crowdsourced service model in a 3D space.

Definition 2: User Trajectory T_u . A user trajectory is the path that is traveled by a user which is a set of k time-stamped samples $T_u = \{\langle ut_i, ux_i, uy_i \rangle, 1 \leq i \leq k\}$.

Definition 3: Spatial Candidate Pair. Given a set of moving services $\mathcal{T} = \{MS_1, MS_2, \dots, MS_n\}$, a user trajectory $T_u = \{up_1, up_2, \dots, up_n\}$ where $up_i = (ux_i, uy_i)$, a search radius r_s , a moving service is formed as a spatial candidate pair cp_{t_i} for a given timestep t_i if its location $MS_k.p_i$ at t_i is inside a disk region $D_{t_i}(\text{center} = up_i(t_i), \text{radius} = r_s)$. The service is inside the disk if the Euclidean or Harvestine distance $d(T_u.up_i, MS_k.p_i)$ between two points of user trajectory $T_u.up_i^{t_i}$ and service trajectory $MS_k.p_i^{t_i}$ at timestep t_i is less than r_s . Without loss of generality, our method can be extended for other distance metrics including network distance and Manhattan distance. The range distance r reflects the maximum spatial proximity allowed.

Definition 4: Valid Candidate Moving Service. A moving service MS_i is a valid candidate service for a given user trajectory if it is paired with the user trajectory over w consecutive timesteps, where $w > 0$, i.e., $C_{MS_i} = \{cp_{t_a}, cp_{t_b}, \dots, cp_{t_w}\}$, $t_a < t_b < \dots < t_w$ and $|a - b| = 1$.

For example, in Fig. 3, we plot six timestep snapshots of a user moving in an arbitrary trajectory. Spatio-temporal neighbour search of a user trajectory at timestep ut_4 is MS_1 and MS_2 , while MS_3 is not a valid candidate moving service. Services in spatial proximity of a user are grouped in circles.

We use the following assumptions in our problem formulation:

- One moving service can only serve one user at any point in time.
- A moving service moves between any two consecutive timestamps t_i and t_{i+1} with a constant speed. As a result, we can determine the position of the moving service at any given time in the time interval $[t_i, t_{i+1}]$. As long as the function of finding the moving service's location is constant time, other speed functions could be considered.
- Radii of all coverage regions of services are fixed to a single value. Therefore, we consider the region around

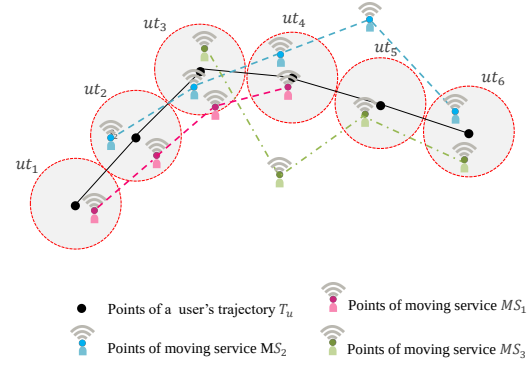


Fig. 3. An example of valid candidate moving services.

- each user's trajectory point to find overlapped services.
- The maximum spatial proximity equals the radius of fixed WiFi hotspot coverage (e.g., 20 m).
- We focus on *deterministic* moving crowdsourced services. A deterministic environment can be achieved using several incentive-based approaches (e.g., [57], [58], [59], [60], [28]). For example, our work in [28] proposes a framework that encourages the movement of crowdsourced IoT service providers from *over-supplied* regions to *under-supplied* regions.

Problem Definition. Given a set of moving crowdsourced services $\mathcal{T} = \{MS_1, MS_2, \dots, MS_n\}$, a user trajectory T_u and a search radius r_s as input, the problem is formulated as finding the “optimal” composition plan that gives the best trade-offs among multiple QoS criteria i.e., a high QoS while maintaining a low number of disconnections. The output is a composition plan CP which is a sequence of moving crowdsourced services $CP = \{S_1, S_2, \dots, S_n\}$ iff S_i is a valid candidate moving service for a given user trajectory (see Definition 4).

3.2 QoS Model for Moving Crowdsourced Service

A key challenge is to find a service that offers a better quality due to the diversity of moving services. QoS parameters are used to distinguish among moving services. It is worth noting that the proposed quality model is extensible. A new QoS parameter (either generic or domain-specific) may be added without fundamentally altering the underlying computation mechanisms. For example, in the crowdsourced energy service scenario, we could use energy-related QoS parameters including Transmission Success Rate and Deliverable Energy Capacity which are proposed in [30] to distinguish among energy services. In our WiFi hotspot scenario, we use one quality attribute that we proposed in [25] *capacity*.

Capacity (cap): Capacity represents the maximum for the information transmission data rate. SNR Shannon-Hartley theorem [61] is used to model the capacity. In our model, the capacity is directly proportional to the *signal strength*. The better signal strength is, the higher Signal-to-Noise-Ratio (SNR) i.e., less error is. We assume that the error rate is fixed. Hence, increasing the capacity increases the signal strength, which leads to more successful transmissions. Given a moving crowdsourced service MS , q_{cap} is

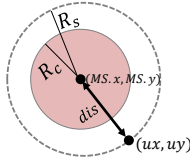


Fig. 4. Strength Model.

computed as follows.

$$q_{cap} = \frac{B}{K} \log_2(1 + str) \quad (1)$$

where B is the total available bandwidth and K is the maximum number of concurrent requests that a moving service MS can support. Total available bandwidth is assumed to be equally allocated between different IoT service users. Signal strength str represents the sensing region of a moving crowdsourced service. The strength is computed based on the distance between the user trajectory point $T_u.p_i = (ux, uy)$ and the moving service point $MS.p_i = (MS.x, MS.y)$ at timestep t_i . For instance, as the user moves closer to the WiFi hotspot, the perceived WiFi signal gets stronger. The str is based on the exponential attenuation probabilistic coverage model [62] which is computed as follows.

$$\begin{cases} 1 & 0 \leq pdis(T_u.p_i, MS.p_i) \leq R_c \\ e^{-kd} & pdis(T_u.p_i, MS.p_i) > R_c \end{cases} \quad (2)$$

where $d = pdis(T_u.p_i, MS.p_i) - R_c$ and $pdis(T_u.p_i, MS.p_i)$ is the perpendicular distance from the crowdsourced service center point $MS.p_i$ to a user trajectory point $T_u.p_i$ (Fig. 4). R_c is a confident radius and k is a decay factor which determines the rate of the signal attenuation with regard to the distance. R_c and k parameters can have different values based on the sensor types and the operation environment which can be obtained through experiments. The strength str is within the range of $(0,1]$. If the user trajectory point is within the distance of R_c , the strength is 1 which means full signal. In the interval $(R_s - R_c)$ where $R_s = pdis(T_u.p_i, MS.p_i)$, the value of str exponentially approaches zero as the perpendicular distance increases. In our model, $str \neq 0$ because a valid candidate crowdsourced service point is paired with a user trajectory point which means that the value of $pdis$ is not beyond R_s (Fig. 4).

The QoS value of a full composite moving crowdsourced IoT service is computed by calculating the average of the capacities of all its component moving services.

4 DEEP REINFORCEMENT LEARNING-BASED COMPOSITION ALGORITHM

The proposed framework aims at finding the optimal service composition for a given moving consumer. The framework neither assumes nor requires consumers to change their trajectories for better services. Simply put, the composition plan is obtained by identifying the set of services that intersects with the consumer's trajectory. Performing the

selection in real-time may disrupt the user experience for consumers. For example, if we assume a consumer that wishes to use a WiFi hotspot for watching videos. The playback may be interrupted every now and then whenever the device is looking for a new service. Therefore, we opt for ahead of time service selection to minimize the interruption overhead.

One single moving service may not fully cover a user trajectory as the user may not necessarily share their entire route with said moving service. As a result, we need to compose candidate moving services to satisfy a user's requirements and ensure service continuity. In this section, we present our approach for selecting and composing all candidate moving services. We leverage *Deep Reinforcement Learning DRL* to find and compose valid candidate services overlapping with the user trajectory. Reinforcement Learning is a subclass of machine learning, where an *agent* learns about an *environment's* behavior through *explorations*. The main reason for our choice to DRL is its ability to discover the "cumulative" optimal set of service trajectories, given the trajectory segments of a user. The DRL does that by assigning rewards for every action the agent invokes. In our work, the actions are the service segments that a user can use. The agent's role is to pick the next service segment that would maximize the overall reward. Therefore, the agent should not only consider the current service and user trajectory segments to make the selection but also future service and user segments. DRL offers a way for the agent to predict the "future" (i.e., unknown knowledge) by learning the behavior of the environment through explorations. The agent is the *composition* algorithm which determines the optimal combination of candidate moving services over time. An environment is characterized by its set of *states* (S) and *actions* (A). In our work, the environment is the combination of service trajectories and a single user trajectory, (see Fig. 5). A user trajectory consists of a series of samples i.e. states. Each sample is a tuple: $\langle ut, ux, uy \rangle$, where ut is the timestamp, ux is the longitude coordinate, and uy is the latitude coordinate. The services trajectories dataset comprises a series of records representing different samples $\langle t, x, y \rangle$ for different services. A sample in the services trajectories dataset contains the same information as a user trajectory sample with the addition of a service ID and QoS attributes (see Definition 1). At any given time, the environment has a current state $s \in S$ i.e. the current user trajectory sample that is reported to the agent. The agent selects an *action* ($a \in A$) i.e., a valid candidate moving IoT service to invoke on the environment. Upon action invocation, the environment generates a *reward* (r) based on the action and its current state. Additionally, the environment changes its state according to the invoked action and previous state. The reward and next state are reported back to the agent i.e., composer.

More formally, we define a composite moving service as follows.

Definition 5: Composite Moving Crowdsourced Service. A composite moving service CS is a sequence of com-

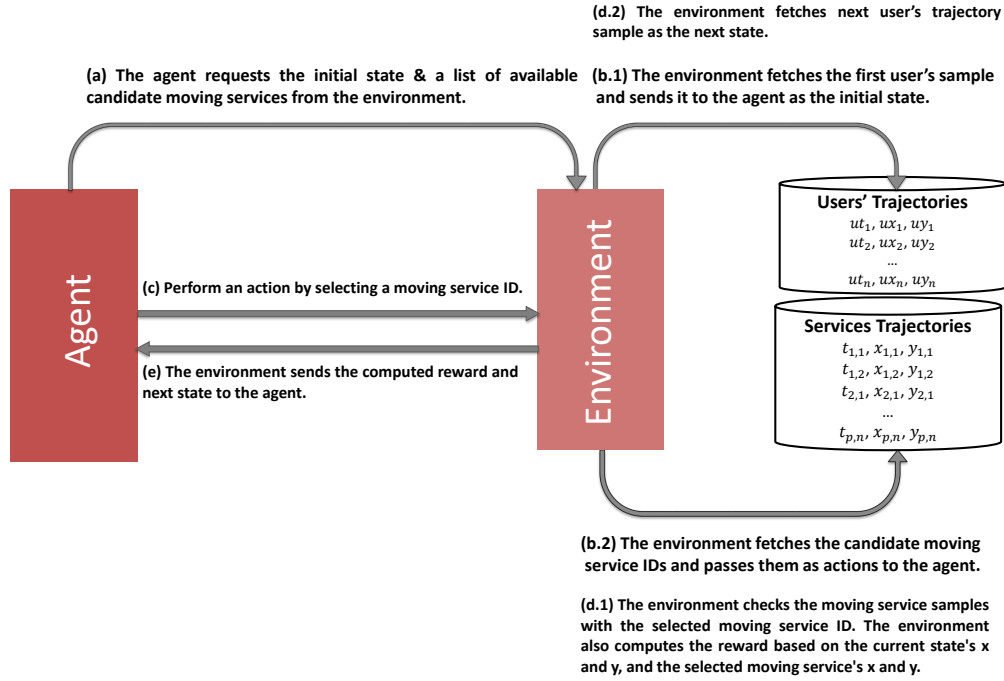


Fig. 5. Reinforcement learning for moving crowdsourced service composition.

ponent moving services, which is defined as a 5-tuple of $\langle S^g, s_0^g, s_r^g, A^g(s), R^g \rangle$ where

- S^g is a finite set of states i.e., samples $\langle t, x, y \rangle$ that are observed by an agent g ;
- $s_0^g \in S^g$ is the initial state of the agent g and the execution of the composite moving service starts from this state. Here the initial state is the first user trajectory sample.
- $s_r^g \subset S^g$ is the set of terminal states where the execution of a composite service terminates upon arriving at one of the states. The terminal state is the last sample of a user trajectory.
- $A^g(s)$ is the set of actions that are taken at each state s . Here we replace the actions with valid candidate moving services (see Definition 4). At each state, we have a set of moving services that could be selected and executed.
- R^g is the reward function when a moving service is invoked. There are multiple QoS objectives which the agent wants to achieve. The user receives the reward i.e., desired QoS when the agent moves to the next state s from s' . Therefore, the reward is computed using the QoS of a composite service.

The composition algorithm aims at finding the *optimal policy* (π^*) which is defined as the procedure for selecting candidate moving services (i.e., the action to be invoked by the agent in each state). This guides the agent toward an optimal set of moving services that gives the best trade-offs among multiple QoS criteria i.e., maximum accumulated reward. More formally, the optimal policy is expressed as

follows:

$$\pi^*(s) = a \quad (3)$$

where $s \in S$ is the environment's state and $a \in A$ is an optimal moving service (i.e., action) to invoke. Obtaining the optimal policy is achieved by solving the *Q-value function*:

$$Q(s, a) = r(s, a) + \gamma \sum_{s'} \max_{a'} Q_i(s', q') | s, a \quad (4)$$

where $Q(s, a)$ is the accumulated QoS reward, given an agent starting at state s and invoking action a , $r(s, a)$ is the instantaneous reward when invoking action a at state s , and γ is a discounting factor.

Finding the optimal policy directly using Equation 4 is impractical due to a potentially large number of states and actions. As a result, the number of state-action combinations increases drastically which in turn leads to exponential time complexity. Therefore, Neural Networks are used in conjunction with Equation 4 to build a policy model. The use of Neural Networks to find the optimal policy is referred to it as *Q-Learning*. The inputs to the Neural Network are the different parameters defining a particular state s , and its outputs are all possible actions that the agent can take.

Q-learning has two main phases: *exploration* and *exploitation*. In the exploration phase, the agent selects random actions and keeps records of the resulting rewards and next states. The collected records are then fed into a Neural Network to train the policy. During the exploitation phase, the agent uses the trained model to predict the next action to take. It is worth noting that in the exploitation phase,

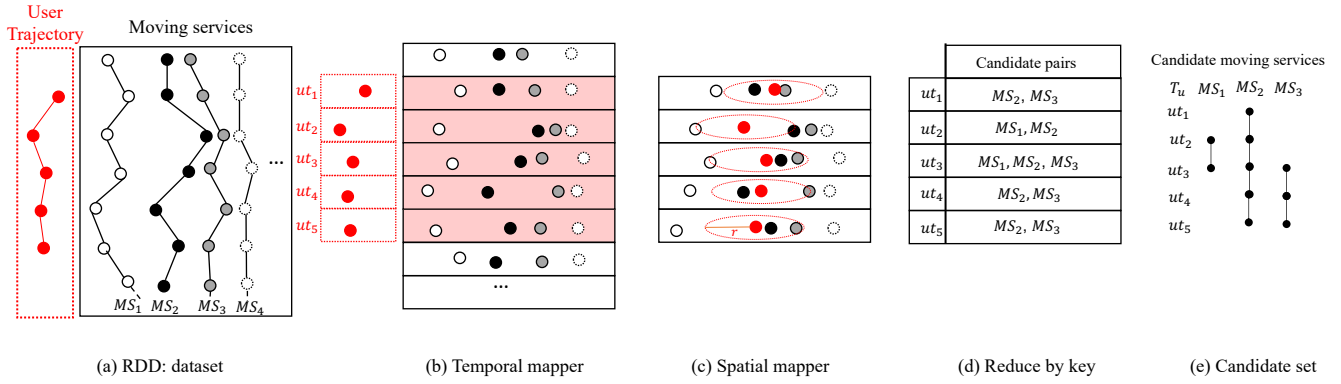


Fig. 6. Workflow of parallel flock-based moving crowdsourced service discovery.

Algorithm 1 Reinforcement Learning-based Composition Training Algorithm

Input: A set of sampled moving service trajectories T_s , a set of sampled user trajectories T_u , and the radius r

Output: A model for policy π

```

1: // Initialization
2:  $\epsilon \leftarrow 1.0$ 
3:  $model \leftarrow$  create an initial neural network model
4:  $memory \leftarrow []$ 
5:  $env \leftarrow$  initialize environment using  $T_s$ 
6: // Model Training
7: for  $t_u \in T_u$  do
8:   for  $i \leftarrow 1$  to repetition do
9:      $env.current\_user = t_u$ 
10:     $state \leftarrow t_u$ 
11:    if  $random() < \epsilon$  then
12:       $action \leftarrow$  pick a random service id from  $T_s$ 
13:    else
14:       $action \leftarrow$  predict the next action using
15:       $model$  and  $state$ 
16:    end if
17:     $reward, next\_state \leftarrow$  act on  $env$  using
18:     $action$ 
19:    store  $next\_state$  and  $reward$  in  $memory$ 
20:  end for
21:  if  $memory$  is full then
22:    train  $model$  using data in  $memory$ 
23:     $\epsilon = \epsilon * 0.995$  // gradually decreasing exploration
24:  end if
25: end for
26: return  $model$ 

```

the agent keeps exploring to adjust the trained model for higher accuracies.

Fig. 5 summarizes the process. An agent starts by requesting the initial state of the environment (step (a)). The environment fetches the first user trajectory sample and reports it as its current state (step (b.1)). Additionally, the environment fetches the list of service IDs and reports them to the agent as possible actions to invoke (step (b.2)). During exploration, the agent randomly invokes an action (by selecting a valid candidate moving service ID) on the environment (step (c)). The environment computes the

reward based on the invoked action. We set the reward as the QoS provided by the selected candidate moving service (step (d.1)). The environment updates its current state by setting it to the next user trajectory sample (step (d.2)). The next state and reward are sent to the agent (step (e)). The agent continues invoking actions until all samples in the user trajectories are visited. Upon traversing all samples of a user trajectory, the environment resets itself (by setting its state to the first user trajectory sample), and the process repeats. Note that several user trajectories should be used to increase the prediction accuracy of the generated model.

Two cases have not been addressed in our previous scenario: (1) no valid candidate moving services for a given user trajectory sample, and (2) an agent selects a moving service that is not a valid candidate (i.e., does not overlap either in time or space with the current user trajectory sample). To resolve the first case, we introduce the concept of a dummy service. A dummy service essentially means that there are no valid candidate moving services for a particular user trajectory sample. The goal of a dummy service is to divert the agent from selecting an invalid candidate moving service when no overlapping services are found. Accomplishing this is carried out by giving a lower reward value whenever the dummy service is selected, e.g., -1 as opposed to $[0 - 1]$ for normal reward values. Resolving the second case is achieved by penalizing the agent when an invalid moving service is selected, e.g., -10. By this, an agent will always favor selecting the dummy service over an invalid one since it has a higher reward value.

Algorithm 1 summarizes the training phase of the approach. The algorithm starts by initializing the parameters (Lines 1-5). Initially, the reinforcement learning agent performs exploration only. The exploration to exploitation ratio is controlled by changing ϵ (1.0 for full exploration and 0.0 for full exploitation). A neural network model is created with random weights in Line 3. The algorithm also initializes an empty space that represents the agent's memory (Line 4). The memory stores information that the agent collects during exploration. More specifically, the memory holds every action the agent takes on the environment. Additionally, it stores the rewards the environment assigns for the taken actions. Finally, the environment is set up using the provided service trajectories T_s (Line 5). The

environment uses the service trajectories T_s to determine its set of possible actions as well as the reward for each taken action.

The model is trained gradually while the agent is exploring the environment and exploiting its current model (Lines 7 through 21). The agent at the beginning favors exploring the environment over exploiting its model. The algorithm loops through each user trajectory in T_s (Line 7). The state of the environment is set using the samples in a given user trajectory. When the agent takes an action, the environment sets its next state to the next sample in the current user trajectory. We allow the environment to exploit each user trajectory *repetition* times (Lines 8 - 10). In other words, a state can be repeated several times. This allows the agent to experiment with different actions given the same state, and observe the rewards associated with each state-action combination. The agent invokes an action on the environment (Lines 11 - 15). The action is the service to choose given a user trajectory sample. The taken action is based on the value of ϵ . The agent tends to take random actions when ϵ has a high value (exploration). Conversely, the agent uses its trained model to decide which action to take. Upon each invoked action, the environment changes its state and returns a reward to the agent (Line 16). The reward is generated based on the QoS of the selected service (i.e., the taken action). The reward and next state values are stored in *memory* (Line 17). The collected actions, states, and rewards in *memory* are used to train the model (Line 20). In other words, *memory* guides the training process towards building a model that favors actions with higher rewards. The value of ϵ is decayed after each training process (Line 21). Decaying ϵ makes the agent use the trained model more, essentially leading the agent to invoke actions that may have higher rewards.

It is worth mentioning that model training and storage is carried out using *edge servers*. We assume edge servers are conveniently set up to be accessed by moving IoT services. Each edge server is responsible for serving a small subset of moving devices. Therefore, storage and processing overheads are negligible.

5 GROUND-TRUTH: PARALLEL FLOCK-BASED MOVING SERVICE DISCOVERY

We propose a *brute-force* approach to find the optimal composition of moving services for a given consumer trajectory. Our approach is used in our experiments to evaluate the accuracy of our deep reinforcement learning-based composition algorithm proposed earlier. In other words, the brute-force approach acts as a *ground-truth* (baseline) to validate the results obtained using the deep reinforcement learning-based algorithm.

The brute-force is the optimal solution as it performs exhaustive search to find all possible co-moving services with a given consumer's trajectory. We achieve this by identifying *co-movement patterns*. Several studies have been proposed to model moving objects including flock [54], [16], convoy [21], swarm [22] and travelling companion

[63]. All of these group movement patterns require the group to contain the same set of individuals during its lifetime [64]. We opt for *the flock pattern* as it is the most appropriate group movement pattern to describe our WiFi hotspot sharing scenario due to the fixed size of the radius (i.e., WiFi hotspot range). However, other co-movement patterns may be more appropriate for other types of applications.

Algorithm 2 Parallel Flock-Based Service Discovery Algorithm

Input: A set of moving services ς , A user travel trajectory T_u , Radius r

Output: A set of spatial candidate pairs

```

1:  $C \leftarrow \{\}$ 
2: compute a list of  $\langle t_i, ST_{t_i} \rangle$  pairs of  $\varsigma$ 
3: compute a list of timesteps  $\langle ut_i, T_{up_i} \rangle$  pairs of  $T_u$ 
4: – Temporal Mapper Phase–
5: for all key-value pairs  $\langle t_i, ST_{t_i} \rangle$  do
6:   left outer join based on timesteps  $ut_i$ 
7: end for
8: – Spatial Mapper Phase –
9: for all  $t_i$  in key-value pairs  $\langle t_i, ST_{t_i} \rangle$  do
10:   find spatial candidate pairs based on  $r$  and  $T_{up_i}$ 
11: end for
12: – Reduce Phase–
13: group-by  $ut_i$ 
14: Return spatial candidate pairs set

```

We deploy a parallel moving service discovery approach. We use Apache Spark as a platform for parallel discovery. The Apache Spark employs MapReduce to handle the scalability and fault tolerance issues. Fig. 6 shows MapReduce jobs which are performed in a sequential workflow. First, the *temporal map phase* conducts temporal pruning of moving services with regards to a user trajectory. In this temporal mapper, timestamp and sampled location are respectively treated as *key* and *value* for each moving service trajectories. Since a user trajectory is important to find the co-movement service, we filter moving service trajectories based on a user trajectory's timesteps. In this regard, we use left outer join as a temporal filtering step to select all moving services that include the user trajectory timesteps (Lines 5-7 Algorithm 2). Second, for each timestep of a user trajectory, the *spatial map phase* is performed over all filtered sub-trajectories of moving services. In the spatial mapper, we retrieve all spatial candidate service pairs (see Definition 2) whose points are inside a range distance r of the user location point in each timestep. Finally, the spatial mapper outputs candidate pairs in each timestep of the user trajectory. It is represented by a list of key-value pairs $\langle ut_i, ST_i \rangle$, where ut_i is the user's timestep and ST_i is a set of candidate moving service IDs that are paired at ut_i (Lines 9-11 Algorithm 2). For each discovered candidate pairs, QoS values are computed. We then extend the discovered candidate pairs to discovered candidate moving services. All candidate moving services

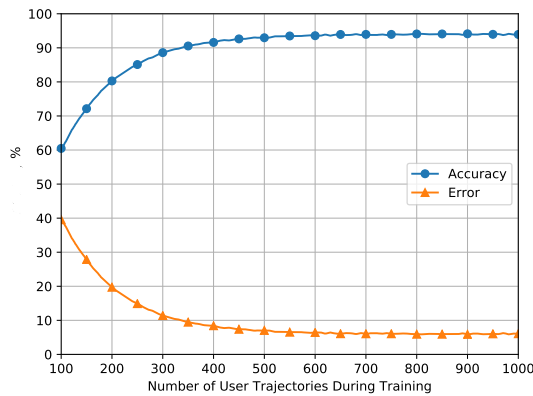


Fig. 7. Accuracy on the indoor dataset

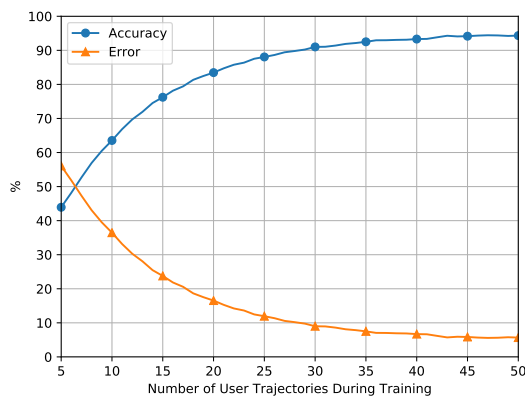


Fig. 8. Accuracy on Illinois dataset.

are validated based on Definition 3 and invalid candidates are disregarded.

6 EXPERIMENTAL EVALUATIONS

We evaluate the accuracy and efficiency of our proposed deep reinforcement learning-based composition algorithm. We utilize our ground-truth approach discussed earlier to validate the results and assess the accuracy of our algorithm. We leverage real pedestrian trajectory datasets throughout our experiments.

6.1 Experiment Setup

All experiments are conducted in a cluster with six nodes on Amazon Web Services. We pick one master node and five slave nodes. The master node has a dual-core processor with 4GB memory. The slave nodes are identical, each equipped with a 16-core processor and 64GB memory. We configure the cluster into 15 Spark executors, each taking 19GB memory and 5 cores.

We use a dense fully connected network to train our model. The inputs to the network are the parameters defining a state whereas the outputs are the actions, which the agent can take. Three hidden layers are used with 512

neurons each. The rectified linear unit (ReLU) activation function is used in all hidden layers. We use dropout with probability 0.5 on all hidden layers to reduce overfitting. The Q-learning's discount factor γ is set to 0.9, whereas the Neural Network's learning rate is 0.001.

The reward function used by the deep learning-based composition algorithm is based on the capacity QoS parameter. We assume that the Q-learning algorithm does not have prior knowledge about QoS attributes of moving services since they are computed based on the distance between a user and a moving service. As a result, we rely on the Q-Learning algorithm to learn the optimal execution policy. We use 70% of the data in each dataset for the training set and the remaining 30% for the test set.

We use two real *pedestrian* trajectory datasets. A single trajectory in the datasets is represented with a series of location samples. Each sample represents the location of a person at a specific time. We use the trajectories in the two datasets to represent WiFi hotspot moving services and user trajectories⁴. The trajectories in the datasets are split into two groups. The first group represents the available WiFi hotspot moving services. The second group is considered as the trajectories of the users. Throughout our experiments, we use our proposed algorithm to perform service selection and composition using the moving services and user trajectories groups. In other words, for a given user trajectory, our proposed algorithm aims at finding a subset of moving services that intersect with the user trajectory while maximizes the QoS. The two datasets are described as follows:

- *Indoor*⁵: The dataset keeps visitors' trajectories in the ATC shopping center in Osaka. The visitors' locations are sampled every 0.03 - 0.06 seconds. In this dataset, we replace sampled timestamps with global sequences that start from 1 to find co-movement patterns. A fixed sampling rate of 0.04 seconds is set, since trajectories do not have synchronized sampled time. We adopt linear interpolation to fill missing points. The dataset contains 1,777,297,164 samples and 185,554 trajectories i.e. moving services.
- *Illinois*⁶: The dataset holds six months trajectories from the daily commute of two members in Argonne National Laboratory of the University of Illinois at Chicago. Each trajectory shows a continuous daily trip of a member in Cook County and/or the Dupage County of Illinois. We treat each trajectory as a moving service. The member's locations are strictly sampled every second. There are 357,706 samples and 207 trajectories i.e. moving services.

To the best of our knowledge, there is limited research investigating QoS-aware moving IoT service composition. We compare the proposed deep reinforcement learning-based composition algorithm to the proposed ground-truth approach to show the accuracy of the selection process.

4. To the best of our knowledge, there are no publicly available crowdsourced hotspot/energy environment datasets.

5. https://irc.atr.jp/crest2010_HRI/ATC_dataset

6. https://www.cs.uic.edu/boxu/mp2p/gps_data.html

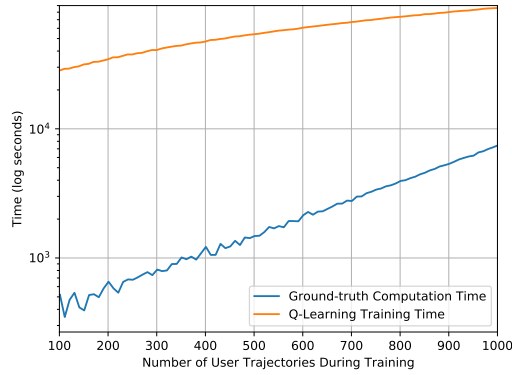


Fig. 9. Computation time vs. No. of user trajectories.

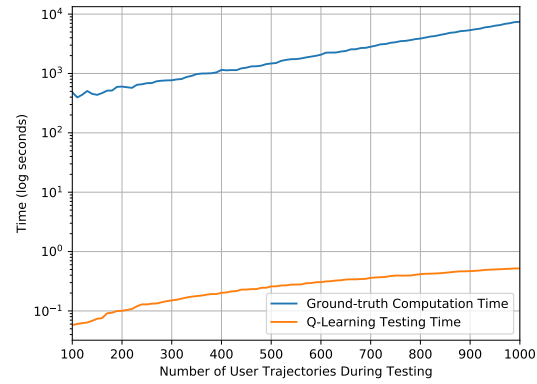


Fig. 10. Computation time vs. No. of user trajectories.

6.2 Experiments results

In this section, we evaluate our deep reinforcement learning-based composition algorithm from three aspects: (1) accuracy; (2) scalability and (3) efficiency of learning.

Accuracy

In the first set of experiments, we study the accuracy of the deep reinforcement learning-based composition algorithm by comparing it to the ground-truth approach on two real datasets. We evaluate the accuracy of the proposed approach by comparing the deep reinforcement learning-based composition algorithm in selecting valid candidate moving services in each timestep and their corresponding valid candidate moving services that are retrieved by the ground-truth. The accuracy is the ratio between the number of times an optimal service was selected cs to the total number of available valid samples ns .

$$Accuracy = \frac{|cs|}{|ns|} \quad (5)$$

Fig. 7 shows the accuracy (blue curve) and error (orange curve) of the deep reinforcement learning-based composition algorithm on indoor dataset while varying the number of user trajectories from 100 to 1,000. As mentioned earlier, the accuracy (and error) results are obtained by validating the proposed approach results with the ground-truth approach, which generates the best possible composition plan (i.e., accuracy 100%). The results show the proposed approach scores high accuracy. As can be seen, the accuracy significantly increases until it reaches around 95% by 500 user trajectories and then it remains stable. As expected, the accuracy is lower when the number of trajectories is low. The reason is that the lower the samples are, the less accurate the result is. Similarly, Fig. 8 shows that the accuracy (blue curve) and error (orange curve) of the composition algorithm on the Illinois dataset. The accuracy increases until it reaches a high accuracy of around 93% after 35 user trajectories.

Scalability

In the second set of experiments, we evaluate the scalability of the proposed approach in terms of training and

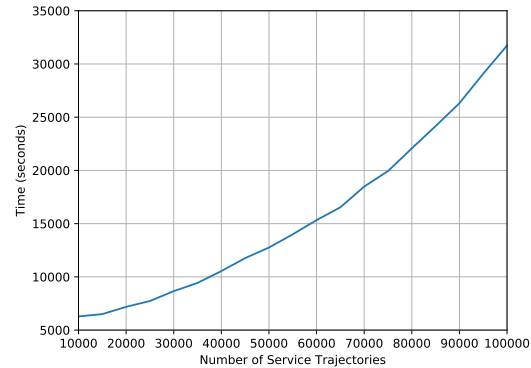


Fig. 11. Convergence time vs. No. of moving services.

selection computation time. Fig. 9 demonstrates the ground-truth computation time in comparison with the Q-learning model training time. As expected, the results show that the ground-truth computation time is significantly lower than the training time. On the other hand, Fig. 10 illustrates the computation time to find valid candidate services of deep reinforcement learning-based composition algorithm significantly outperforms the ground-truth (i.e. less than 0.1 Sec in Q-learning in comparison with close to 10,000 Sec in ground-truth for 1000 services). This indicates that the deep reinforcement learning-based composition algorithm can select valid candidates much faster than the spatio-temporal MapReduce approach.

Efficiency of learning

In the third set of experiments, we study the learning speed with increasing the number of moving services on indoor and Illinois datasets. Firstly, we vary the number of moving services from 10,000 to 100,000 on the indoor dataset. The results in Fig. 11 illustrate how fast the algorithm converges to the optimal composition plan during the learning phase. The convergence time increases polynomially with the increasing number of moving services which is an expected result. This is because the number of candidate moving

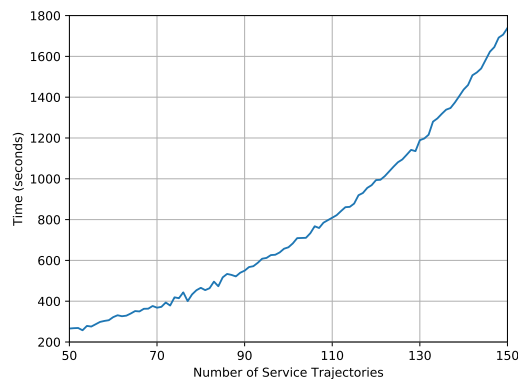


Fig. 12. Convergence time vs. No. of moving services.

services that should be searched at each state increases exponentially. Additionally, the convergence time increases slower than the number of moving services. Secondly, we vary the number of moving services from 50 to 150 on the Illinois dataset. Fig. 12 shows that the convergence time increases with increasing the number of moving services. The results also show that our model trains relatively fast to find optimal composition plans (e.g., less than 30 min for 150 services).

7 CONCLUSION

We proposed a crowdsourced IoT service framework to select and compose moving crowdsourced IoT services based on spatio-temporal factors. We developed a deep reinforcement learning-based algorithm to select and compose moving services considering QoS parameters without using an index. We also developed a spatio-temporal MapReduce based on flock patterns using Apache Spark to discover moving services as a ground-truth. Our experiments show the scalability and high accuracy of the proposed approach in comparison with the ground-truth. In our future work, we develop and test our proposed approach on different motion patterns, i.e., transportation modes. We also plan to extend the proposed framework to temporal non-deterministic moving services.

REFERENCES

- [1] E. L. Nekaj, "The crowd economy: From the crowd to businesses to public administrations and multinational companies," in *Crowdfunding for Sustainable Entrepreneurship and Innovation*. IGI Global, 2017, pp. 1–19.
- [2] A. Taeihagh, "Crowdsourcing: a new tool for policy-making?" *Policy Sciences*, vol. 50, no. 4, pp. 629–647, Dec 2017. [Online]. Available: <https://doi.org/10.1007/s11077-017-9303-3>
- [3] J. Howe, "The rise of crowdsourcing," *Wired magazine*, vol. 14, no. 6, pp. 1–4, 2006.
- [4] J. A. Burke, D. Estrin, M. Hansen, A. Parker, N. Ramanathan, S. Reddy, and M. B. Srivastava, "Participatory sensing," *Center for Embedded Network Sensing*, 2006.
- [5] L. Kazemi and C. Shahabi, "Geocrowd: enabling query answering with spatial crowdsourcing," in *Proceedings of the 20th International Conference on Advances in Geographic Information Systems*. ACM, 2012, pp. 189–198.

- [6] A. Bouguettaya, M. Singh, M. Huhns, Q. Z. Sheng, H. Dong, Q. Yu, A. G. Neiat, S. Mistry, B. Benatallah, B. Medjahed *et al.*, "A service computing manifesto: the next 10 years," *Communications of the ACM*, vol. 60, no. 4, pp. 64–72, 2017.
- [7] E. Badidi, Y. Atif, Q. Z. Sheng, and M. Maheswaran, "On personalized cloud service provisioning for mobile users using adaptive and context-aware service composition," *Computing*, vol. 101, no. 4, pp. 291–318, 2019.
- [8] X. Peng, J. Gu, T. H. Tan, J. Sun, Y. Yu, B. Nuseibeh, and W. Zhao, "Crowdsourcing: optimizing mobile crowdsourcing and service composition," *ACM Transactions on Internet Technology (TOIT)*, vol. 18, no. 2, pp. 1–25, 2018.
- [9] S. Deng, L. Huang, J. Taheri, J. Yin, M. Zhou, and A. Y. Zomaya, "Mobility-aware service composition in mobile communities," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 47, no. 3, pp. 555–568, 2017.
- [10] P. K. D. Pramanik and P. Choudhury, "Mobility-aware service provisioning for delay tolerant applications in a mobile crowd computing environment," *SN Applied Sciences*, vol. 2, no. 3, pp. 1–17, 2020.
- [11] S. Deng, L. Huang, D. Hu, J. L. Zhao, and Z. Wu, "Mobility-enabled service selection for composite services," *IEEE Transactions on Services Computing*, vol. 9, no. 3, pp. 394–407, 2016.
- [12] Z. Zhou, D. Zhao, L. Liu, and P. C. Hung, "Energy-aware composition for wireless sensor networks as a service," *Future Generation Computer Systems*, vol. 80, pp. 299–310, 2018.
- [13] M. Sun, Z. Zhou, and Y. Duan, "Energy-aware service composition of configurable iot smart things," in *2018 14th International Conference on Mobile Ad-Hoc and Sensor Networks (MSN)*. IEEE, 2018, pp. 37–42.
- [14] H. Wu, S. Deng, W. Li, J. Yin, X. Li, Z. Feng, and A. Y. Zomaya, "Mobility-aware service selection in mobile edge computing systems," in *2019 IEEE International Conference on Web Services (ICWS)*. IEEE, 2019, pp. 201–208.
- [15] A. Lakhdari, A. Bouguettaya, S. Mistry, and A. G. Neiat, "Composing energy services in a crowdsourced iot environment," *IEEE Transactions on Services Computing*, 2020.
- [16] J. Gudmundsson and M. van Kreveld, "Computing longest duration flocks in trajectory data," in *Proceedings of the 14th annual ACM international symposium on Advances in geographic information systems*. ACM, 2006, pp. 35–42.
- [17] R. Vernica, M. J. Carey, and C. Li, "Efficient parallel set-similarity joins using mapreduce," in *the 2010 ACM International Conference on Management of data SIGMOD*. ACM, 2010, pp. 495–506.
- [18] X. Zhang, L. Chen, and M. Wang, "Efficient multi-way theta-join processing using mapreduce," *Proceedings of the VLDB Endowment*, vol. 5, no. 11, pp. 1184–1195, 2012.
- [19] Y. Fang, R. Cheng, W. Tang, S. Maniu, and X. Yang, "Scalable algorithms for nearest-neighbor joins on big trajectory data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 3, pp. 785–800, 2016.
- [20] Q. Fan, D. Zhang, H. Wu, and K.-L. Tan, "A general and parallel platform for mining co-movement patterns over large-scale trajectories," *Proceedings of the VLDB Endowment*, vol. 10, no. 4, pp. 313–324, 2016.
- [21] H. Jeung, M. L. Yiu, X. Zhou, C. S. Jensen, and H. T. Shen, "Discovery of convoys in trajectory databases," *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 1068–1080, 2008.
- [22] Z. Li, B. Ding, J. Han, and R. Kays, "Swarm: Mining relaxed temporal moving object clusters," *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 723–734, 2010.
- [23] Y. Wang, E.-P. Lim, and S.-Y. Hwang, "Efficient mining of group patterns from user movement data," *Data & Knowledge Engineering*, vol. 57, no. 3, pp. 240–282, 2006.
- [24] Y. Li, J. Bailey, and L. Kulik, "Efficient mining of platoon patterns in trajectory databases," *Data & Knowledge Engineering*, vol. 100, pp. 167–187, 2015.
- [25] A. G. Neiat, A. Bouguettaya, T. Sellis, and S. Mistry, "Crowdsourced coverage as a service: two-level composition of sensor cloud services," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 7, pp. 1384–1397, 2017.
- [26] A. G. Neiat, A. Bouguettaya, and T. Sellis, "Spatio-temporal composition of crowdsourced services," in *International Conference on Service-Oriented Computing (ICSOC)*. Springer, 2015, pp. 373–382.
- [27] S. Seneviratne, Y. Hu, T. Nguyen, G. Lan, S. Khalifa, K. Thilakarathna, M. Hassan, and A. Seneviratne, "A survey of wearable

- devices and challenges,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2573–2620, 2017.
- [28] A. G. Neiat, A. Bouguettaya, and S. Mistry, “Incentive-based crowdsourcing of hotspot services,” *ACM Transactions on Internet Technology (TOIT)*, vol. 19, no. 1, pp. 1–24, 2019.
- [29] W. Na, J. Park, C. Lee, K. Park, J. Kim, and S. Cho, “Energy-efficient mobile charging for wireless power transfer in internet of things networks,” *IEEE Internet of Things Journal*, vol. 5, no. 1, pp. 79–92, 2018.
- [30] A. Lakhdari, A. Bouguettaya, and A. G. Neiat, “Crowdsourcing energy as a service,” in *International Conference on Service-Oriented Computing*. Springer, 2018, pp. 342–351.
- [31] Y. Tong, Z. Zhou, Y. Zeng, L. Chen, and C. Shahabi, “Spatial crowdsourcing: a survey,” *The VLDB Journal*, vol. 29, no. 1, pp. 217–250, 2020.
- [32] D. Shin, S. M. Arisona, S. Georgakopoulou, G. Schmitt, and S. Kim, “A crowdsourcing urban simulation platform on smartphone technology: Strategies for urban data visualization and transportation mode detection,” in *Proceedings of the 30th eCAADe Conference*, 2012, pp. 377–384.
- [33] G. Marzano, J. Lizut, and L. O. Siguencia, “Crowdsourcing solutions for supporting urban mobility,” *Procedia Computer Science*, vol. 149, pp. 542–547, 2019.
- [34] Y. Chon, N. D. Lane, F. Li, H. Cha, and F. Zhao, “Automatically characterizing places with opportunistic crowdsensing using smartphones,” in *Proceedings of the 2012 ACM Conference on Ubiquitous Computing*. ACM, 2012, pp. 481–490.
- [35] M. F. Bulut, Y. S. Yilmaz, and M. Demirbas, “Crowdsourcing location-based queries,” in *2011 IEEE International Conference on Pervasive Computing and Communications Workshops (PERCOM Workshops)*. IEEE, 2011, pp. 513–518.
- [36] J. Zimmerman, A. Tomasic, C. Garrod, D. Yoo, C. Hiruncharoenvate, R. Aziz, N. R. Thiruvengadam, Y. Huang, and A. Steinfeld, “Field trial of tiramisu: crowd-sourcing bus arrival times to spur co-design,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 2011, pp. 1677–1686.
- [37] A. Steinfeld, J. Zimmerman, A. Tomasic, D. Yoo, and R. Aziz, “Mobile transit information from universal design and crowdsourcing,” *Transportation Research Record: Journal of the Transportation Research Board*, no. 2217, pp. 95–102, 2011.
- [38] B. Ferris, K. Watkins, and A. Borning, “Onebusaway: results from providing real-time arrival information for public transit,” in *Proceedings of the Conference on Human Factors in Computing Systems (SIGCHI)*. ACM, 2010, pp. 1807–1816.
- [39] A. B. Said, A. Erradi, A. Ghari Neiat, and A. Bouguettaya, “Mobile crowdsourced sensors selection for journey services,” in *International Conference on Service-Oriented Computing*. Springer, 2018, pp. 463–477.
- [40] B. Kantarci and H. T. Mouftah, “Sensing services in cloud-centric internet of things: A survey, taxonomy and challenges,” in *2015 IEEE International Conference on Communication Workshop (ICCW)*. IEEE, 2015, pp. 1865–1870.
- [41] A. Murturi, B. Kantarci, and S. F. Oktug, “A reference model for crowdsourcing as a service,” in *2015 IEEE 4th International Conference on Cloud Networking*. IEEE, 2015, pp. 64–66.
- [42] G. Iosifidis, L. Gao, J. Huang, and L. Tassiulas, “Enabling crowd-sourced mobile internet access,” in *IEEE International Conference on Computer Communications INFOCOM*, 2014, pp. 451–459.
- [43] X. Sheng, X. Xiao, J. Tang, and G. Xue, “Sensing as a service: A cloud computing system for mobile phone sensing,” in *Sensors 2012*. IEEE, 2012, pp. 1–4.
- [44] X. Sheng, J. Tang, X. Xiao, and G. Xue, “Sensing as a service: Challenges, solutions and future directions,” *IEEE Sensors Journal*, vol. 13, no. 10, pp. 3733–3741, 2013.
- [45] X. Peng, J. Gu, T. H. Tan, J. Sun, Y. Yu, B. Nuseibeh, and W. Zhao, “Crowdservice: Serving the individuals through mobile crowdsourcing and service composition,” in *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2016, pp. 214–219.
- [46] H. Wang and X. Wang, “A novel approach to large-scale services composition,” in *Asia-Pacific Web Conference*. Springer, 2013, pp. 220–227.
- [47] W. Xu, J. Cao, H. Zhao, and L. Wang, “A multi-agent learning model for service composition,” in *2012 IEEE Asia-Pacific Services Computing Conference (APSCC)*. IEEE, 2012, pp. 70–75.
- [48] K. Gai and M. Qiu, “Reinforcement learning-based content-centric services in mobile sensing,” *IEEE Network*, vol. 32, no. 4, pp. 34–39, 2018.
- [49] I. J. Jureta, S. Faulkner, Y. Achbany, and M. Saerens, “Dynamic web service composition within a service-oriented architecture,” in *IEEE International Conference on Web Services, ICWS 2007*. IEEE, 2007, pp. 304–311.
- [50] Y. Achbany, F. Fouss, L. Yen, A. Pirotte, and M. Saerens, “Optimal tuning of continual online exploration in reinforcement learning,” in *International Conference on Artificial Neural Networks*. Springer, 2006, pp. 790–800.
- [51] H. Wang and P. Tang, “Preference-aware web service composition by reinforcement learning,” in *20th IEEE International Conference on Tools with Artificial Intelligence, ICTAI’08*, vol. 2. IEEE, 2008, pp. 379–386.
- [52] A. Moustafa and M. Zhang, “Multi-objective service composition using reinforcement learning,” in *International Conference on Service-Oriented Computing*. Springer, 2013, pp. 298–312.
- [53] H. Wang, Q. Wu, X. Chen, Q. Yu, Z. Zheng, and A. Bouguettaya, “Adaptive and dynamic service composition via multi-agent reinforcement learning,” in *2014 IEEE International Conference on Web Services (ICWS)*. IEEE, 2014, pp. 447–454.
- [54] M. R. Vieira, P. Bakalov, and V. J. Tsotras, “On-line discovery of flock patterns in spatio-temporal data,” in *Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, 2009, pp. 286–295.
- [55] P. Kalnis, N. Mamoulis, and S. Bakiras, “On discovering moving clusters in spatio-temporal data,” in *International Symposium on Spatial and Temporal Databases*. Springer, 2005, pp. 364–381.
- [56] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Kdd*, vol. 96, no. 34, 1996, pp. 226–231.
- [57] D. Zhao, H. Wang, H. Ma, H. Xu, L. Liu, and P. Zhang, “Crowdolr: Toward object location recognition with crowdsourced fingerprints using smartphones,” *IEEE Transactions on Human-Machine Systems*, vol. 47, no. 6, pp. 1005–1016, 2017.
- [58] H. Wang, D. Zhao, H. Ma, and L. Ding, “Min-max planning of time-sensitive and heterogeneous tasks in mobile crowd sensing,” in *2018 IEEE Global Communications Conference (GLOBECOM)*. IEEE, 2018, pp. 1–7.
- [59] M. C. Bliemer, M. Dicke-Ogenia, and D. Ettema, “Rewarding for avoiding the peak period: a synthesis of four studies in the netherlands,” 2010.
- [60] P. Cohen, R. Hahn, J. Hall, S. Levitt, and R. Metcalfe, “Using big data to estimate consumer surplus: The case of uber,” National Bureau of Economic Research, Tech. Rep., 2016.
- [61] C. E. Shannon, “A mathematical theory of communication,” *ACM SIGMOBILE Mobile Computing and Communications Review*, vol. 5, no. 1, pp. 3–55, 2001.
- [62] İ. K. Altunel, N. Aras, E. Güney, and C. Ersoy, “Binary integer programming formulation and heuristics for differentiated coverage in heterogeneous sensor networks,” *Computer Networks*, vol. 52, no. 12, pp. 2419–2431, 2008.
- [63] L.-A. Tang, Y. Zheng, J. Yuan, J. Han, A. Leung, C.-C. Hung, and W.-C. Peng, “On discovery of traveling companions from streaming trajectories,” in *2012 IEEE 28th International Conference Data Engineering (ICDE) on*. IEEE, 2012, pp. 186–197.
- [64] E. Naserian, X. Wang, X. Xu, and Y. Dong, “A framework of loose travelling companion discovery from human trajectories,” *IEEE Transactions on Mobile Computing*, vol. 17, no. 11, pp. 2497–2511, 2018.



Azadeh Ghari Neiat is a lecturer in the School of Information Technology at the Deakin University. Before joining Deakin University, she was a postdoctoral research fellow at the University of Sydney since 2017. She was awarded a PhD in computer science at RMIT University, Australia in 2017. She has published in top journals and conferences such as CACM, IEEE TKDE, IEEE TSC, ACM TOIT, Future Generation Computer Systems, ICSOC, ICWS, MobiQuitous etc. Her research interest lies at the intersections of Mobile Crowdsourcing, AI, IoT, and Spatio-Temporal Data Analysis.



Athman Bouguettaya is Professor and Head of School of Computer Science at the University of Sydney, Sydney, Australia. He received his PhD in Computer Science from the University of Colorado at Boulder (USA) in 1992. He is or has been on the editorial boards of several journals including, the IEEE Transactions on Services Computing, ACM Transactions on Internet Technology, the International Journal on Next Generation Computing, VLDB Journal, Distributed and Parallel Databases Journal, and the International Journal of Cooperative Information Systems. He has published more than 200 books, book chapters, and articles in journals and conferences in the area of databases and service computing. He is a Fellow of the IEEE and a Distinguished Scientist of the ACM.



Mohammed Bahutair is a PhD student in the School of Computer Science at the University of Sydney, Australia. He received his bachelor degree in Computer Engineering from Ittihad University, UAE 2012 and his Masters degree in Computer Engineering from University of Sharjah, UAE 2015. His research interests are Machine Learning, Trust, IoT and Big Data Mining.